

TARAflow Referenzfall: CNC-Fertigungssystem

© Jürgen Messerer · 2026 · Alle Rechte vorbehalten

Zweck dieses Dokuments: Demonstration der TARAflow-Methodik anhand eines realen OT-Systems. Der Referenzfall zeigt die Stärken des graphbasierten Ansatzes. Insbesondere die Integration von Safety und Security, die automatische Diagrammgenerierung und die vollständige Rückverfolgbarkeit von der Systemmodellierung bis zur Risikobewertung.

0. Einführung: Warum TARAflow?

Das Problem klassischer Methoden

Klassische Threat-Modeling-Ansätze, ob STRIDE-naiv oder regelbasierte Tools wie IriusRisk, analysieren **Elemente**, nicht **Auswirkungen**. Das Ergebnis ist eine Liste generischer Threats ohne konkreten Schadensbezug:

```
Klassisch:  
CNC Controller → Tampering (möglich)  
                  Information Disclosure (möglich)  
                  Denial of Service (möglich)
```

Man weiss was angegriffen werden kann, aber nicht warum es relevant ist, wie schwer der Schaden wäre und was die Safety-Konsequenz für Menschen ist. Jedes Element generiert dieselbe STRIDE-Liste, unabhängig davon ob es kritisch ist oder nicht. Der Analyseaufwand ist hoch, die Aussagekraft gering.

Der TARAflow-Ansatz

TARAflow dreht die Logik um: **Assets und ihr Impact steuern die Analyse**, nicht Elementtypen und Regelkataloge.

```
TARAflow:  
DFD-Element → Asset → Impact + CIANAAA-Schutzziele → STRIDE → Risk  
              ↑  
              Was ist schützenswert?      (Asset-Kategorie)  
              Wie hoch ist der Schaden?    (Impact-Bewertung)  
              Welches Schutzziel wird verletzt? (C/I/A/N/A/A/A)
```

Das Ergebnis ist keine generische Threat-Liste, sondern eine vollständige, nachvollziehbare Kette von der Systemstruktur bis zur Konsequenz. CIANAAA steht für Confidentiality, Integrity, Availability, Non-Repudiation, Authentication, Authorization und Accountability. TARAflow unterscheidet insbesondere zwischen Accountability (systeminterne Nachvollziehbarkeit) und Non-Repudiation (beweisfeste Nicht-Abstreitbarkeit). Diese Unterscheidung wird im Methodenkern formal definiert.

Drei konkrete Gewinne gegenüber klassischen Methoden

- 1. Vollständige Rückverfolgbarkeit** Jede Massnahme ist bis zum Asset, zur Schutzziel-Verletzung und zur regulatorischen Anforderung zurückverfolgbar. Das ist kein Nice-to-have. Es ist eine Anforderung nach IEC 62443-4-1 und EU Cyber Resilience Act.
- 2. Safety + Security in einem Modell** Safety-Konsequenzen (Personengefährdung) werden direkt aus dem Security-Modell abgeleitet, ohne separates Safety-Dokument, ohne Doppelpflege, ohne Inkonsistenz zwischen zwei Modellen.
- 3. Fokus durch risikoorientierte Priorisierung** Nicht jedes Element wird gleich tief analysiert. Asset-Kritikalität steuert die STRIDE-Tiefe proportional zum potenziellen Schaden. Das reduziert Analyseaufwand ohne Vollständigkeit zu opfern.

Positionierung

TARAflow richtet sich an **Hersteller** die eine normkonforme TARA mit Nachweispflicht nach IEC 62443-4-1/4-2, ISO 21434, EN 50742 oder dem EU Cyber Resilience Act durchführen müssen. Es ist kein Tool für schnelle erste Übersichten, sondern für begründete, auditierbare Sicherheitsanalysen.

1. Systemkontext

Ein CNC-Fertigungssystem besteht aus einer oder mehreren CNC-Maschinen, einem Robotersystem für die Werkstückhandhabung, einem MES (eigenentwickelte Steuerungsapplikation mit Web-Frontend und gRPC-Schnittstellen), einem SCADA System sowie verschiedenen Akteuren wie Bediener, Programmierer und Remote-Support. Das System hat direkte Sicherheitsrelevanz, da unkontrollierte Maschinenbewegungen zu schweren Verletzungen oder Tod führen können.

Systemgrenze: Das System unter Analyse umfasst MES, SCADA System, CNC Controller, Robot Controller und Safety Controller (SIS) inklusive aller Schnittstellen nach aussen. Der VPN Gateway ist Teil der Systemgrenze als kontrollierter Zugangspunkt für Remote Support. Externe Akteure (Remote Support, NC Programmer) sowie Kundensysteme ausserhalb der deklarierten Trust Boundaries liegen ausserhalb der Systemgrenze oder an deren Rand.

Regulatorischer Kontext: IEC 62443-4-1 (SDL), EN 50742 (Safety), ISO 12100 (Maschinensicherheit), EU Cyber Resilience Act.

2. DFD-Modellierung

2.1 Elemente

Trust Boundaries:

- – **Customer Network Area [CNA]** – Systemgrenze (alle internen Elemente sowie Akteure innerhalb des Kundennetzes)
- – **ERP-Zone [ERP]** – IT-Netzwerk mit MES und Datenhaltung; Eintritt erfordert Authentifizierung
- – **Machine Control Area [MCA]** – Physischer Maschinenbereich mit SCADA und Feldgeräten
- – **Safety Boundary Area [SBA]** – SIS-Zone, safety-kritische Komponenten

Hierarchie: CNA ist die äussere Systemgrenze. ERP, MCA und SBA sind gleichrangige Zonen innerhalb von CNA. Ein Dataflow darf maximal eine Trust Boundary überqueren.

External Entities:

- ◦ **Remote Support [ext]** – Hersteller-Fernwartung via VPN
- ◦ **NC Programmer [CNA]** – NC-Programm Erstellung und Upload; innerhalb CNA, ausserhalb ERP-Zone
- ◦ **Operator [MCA]** – Maschinenbediener vor Ort
- ◦ **Maintenance Technician [MCA]** – Physische Wartung vor Ort

Processes:

- □ **VPN Gateway [CNA]** – Kontrollierter Zugangspunkt für Remote Support; einziger Einstiegspunkt von ausserhalb CNA
- □ **MES [ERP]** – Eigenentwickelte Steuerungsapplikation (Web-Frontend, .NET Backend, gRPC-Schnittstellen)
- □ **SCADA System [MCA]** – HMI, Überwachung, Auftragssteuerung, Datenerfassung; einzige Kommunikationsschicht zu Feldgeräten, gRPC-Client
- □ **CNC Controller [MCA]** – Ausführung von NC-Programmen, Achsregelung, Filesystem-Zugriff
- □ **Robot Controller [MCA]** – Bewegungsplanung, Werkstückhandhabung, Sensorik
- □ **Safety Controller (SIS) [SBA]** – Not-Halt, Schutzzaunüberwachung, PLC-Speicher

Data Stores (Speichersysteme):

- 🗄 **Network Share [ERP]** – Freigegebener Ordner für NC-Programm Upload
- 🗄 **Historian Database [MCA]** – Zeitreihendatenbank (Produktions- und Prozessdaten)
- 🗄 **CNC Filesystem [MCA]** – Lokaler Speicher des CNC Controller (NC-Programme, Kalibrierungsdaten)
- 🗄 **PLC Memory [SBA]** – Interner Speicher des Safety Controller (Safety-Parameter)

Data Flows (nach TARAflow Naming-Konvention):

- → pull scada diagnostics ext [req] – Remote Support → VPN Gateway
- → pull scada diagnostics ext [resp] – VPN Gateway → Remote Support
- → pull scada diagnostics int [req] – VPN Gateway → SCADA System
- → pull scada diagnostics int [resp] – SCADA System → VPN Gateway
- → push nc program [cmd] – NC Programmer → MES
- → write nc program [cmd] – MES → Network Share
- → pull nc program [req] – SCADA System → MES
- → pull nc program [resp] – MES → SCADA System
- → read nc program [req] – MES → Network Share
- → read nc program [resp] – → Network Share -> MES
- → push order data [cmd] – MES → SCADA System
- → stream process data [stream] – SCADA System → MES
- → write calibration data [cmd] – Maintenance Technician → SCADA System
- → send cmd manual input [cmd] – Operator → SCADA System
- → read nc program [req] – CNC Controller → SCADA System
- → read nc program [resp] – SCADA System → CNC Controller
- → send cnc machine commands [cmd] – SCADA System → CNC Controller
- → stream machine status [stream] – CNC Controller → SCADA System
-
- → write calibration data int [cmd] – SCADA System → CNC Controller
- → pull cnc diagnostics [req] – SCADA System → CNC Controller
- → pull cnc diagnostics [resp] – CNC Controller → SCADA System
-
- → read robot program [req] – Robot Controller → SCADA System
- → read robot program [resp] – SCADA System → Robot Controller
- → send cmd robot control [cmd] – SCADA System → Robot Controller
- → stream robot status [stream] – Robot Controller → SCADA System
- → pull robot diagnostics [req] – SCADA System → Robot Controller
- → pull robot diagnostics [resp] – Robot Controller → SCADA System
- → write production data [cmd] – SCADA System → Historian Database
- → read calibration data [req] – CNC Controller → CNC Filesystem
- → read calibration data [resp] – CNC Filesystem → CNC Controller
- → write calibration data store [cmd] – CNC Controller → CNC Filesystem
- → write safety params [cmd] – Maintenance Technician → Safety Controller (SIS)
- → write safety params int [cmd] – Safety Controller (SIS) → PLC Memory
- → read safety params [req] – Safety Controller (SIS) → PLC Memory

- → read safety params [resp] – PLC Memory → Safety Controller (SIS)
- → send cnc emergency stop [cmd] – Safety Controller (SIS) → CNC Controller
- → send robot emergency stop [cmd] – Safety Controller (SIS) → Robot Controller
- → stream sis status [stream] – Safety Controller (SIS) → SCADA System

3. Asset-Modellierung und Beziehungen

TARAflow verwendet fünf Asset-Kategorien die gemeinsam alle schützenswerten Güter eines Systems abdecken. Die Beziehungen zwischen DFD-Elementen und Assets sind typisiert. Sie beschreiben präzise wie ein Element ein Asset berührt. Das ist die Grundlage für die assetbasierte STRIDE-Analyse.

Safety Annotation Layer: Safety-Aspekte werden als optionaler Annotation Layer auf bestehenden Beziehungen modelliert – nicht als separates Safety-Modell. Wird an einer Beziehung eine SafetyAnnotation gesetzt (**relevance: 'direct', impact: 'fatality'**), signalisiert der Analyst dass dieses Asset über diese Beziehung direkt safety-relevant ist. Die Annotation ist der primäre Trigger für den Safety Impact im Asset-Tab. Der Safety Impact am Asset kann abgeleitet (**derived**) oder manuell (**manual**) gesetzt werden. Bei manueller Setzung ist eine Begründung (**rationale**) Pflicht.

3.1 Data Assets

Identifizierte Data Assets: Fertigungsrezepte, Kalibrierungsdaten, Produktionsdaten, Maschinenstatus, Diagnosedaten, Safety-Parameter, Auftragsdaten.

Beziehungen:

```

DataStore "CNC Filesystem"
├ stores → Data Asset "Fertigungsrezepte"
└ stores → Data Asset "Kalibrierungsdaten"
    └ safety: { relevance: 'direct', impact: 'irreversible_injury',
                rationale: 'Falsche Kalibrierung → unkontrollierte Bewegung' }

DataStore "Network Share"
└ stores → Data Asset "Fertigungsrezepte"

DataStore "Historian Database"
└ stores → Data Asset "Produktionsdaten"

DataStore "PLC Memory"
├ stores → Data Asset "Safety-Parameter"
└ safety: { relevance: 'indirect', impact: 'fatality',
            rationale: 'Manipulation der Safety-Parameter kompromittiert SIS-
Logik -                               indirekter Einfluss über Safety Controller auf Not-Halt-
Fähigkeit' }

Process "MES"
├ creates → Data Asset "Auftragsdaten"
└ stores → Data Asset "Fertigungsrezepte"

Process "CNC Controller"
├ reads → Data Asset "Fertigungsrezepte"
├ reads → Data Asset "Kalibrierungsdaten"
└ modifies → Data Asset "Maschinenstatus"

Process "SCADA System"
├ creates → Data Asset "Produktionsdaten"
└ reads → Data Asset "Fertigungsrezepte"

DF "push nc program [cmd]"
└ transports → Data Asset "Fertigungsrezepte"
    └ safety: { relevance: 'direct', impact: 'fatality',

```

```
        rationale: 'Manipuliertes NC-Programm → falsche Werkzeugbewegung →  
Personengefährdung' }
```

```
DF "pull scada diagnostics ext [resp]"  
└ transports → Data Asset "Diagnosedaten"
```

```
DF "pull cnc diagnostics [resp]"  
└ transports → Data Asset "Diagnosedaten"
```

3.2 Function Assets

Identifizierte Function Assets: Not-Halt-Funktion, NC-Programm-Ausführung.

Function Asset vs. Process Asset: Function Assets beschreiben *was das System können muss* (Capability). Process Assets beschreiben *wie es zur Laufzeit abläuft* (aktiver Ablauf mit Sequenz und Zustand). Die Not-Halt-Funktion ist eine Capability – der Not-Halt-Prozess ist ihre Laufzeit-Instanz.

Beziehungen:

```
DF "send cnc emergency stop [cmd]"  
└ is_an → Function Asset "Not-Halt-Funktion"  
  └ safety: { relevance: 'direct', impact: 'fatality',  
              rationale: 'Unterbrechung oder Manipulation des Emergency-Stop-  
Signals  
verhindert zuverlässiges Stoppen der Maschine' }  
  
DF "send robot emergency stop [cmd]"  
└ is_an → Function Asset "Not-Halt-Funktion"  
  └ safety: { relevance: 'direct', impact: 'fatality',  
              rationale: 'identisch – gilt für Robot Controller' }  
  
Process "Safety Controller (SIS)"  
└ executes → Function Asset "Not-Halt-Funktion"  
  └ safety: { relevance: 'direct', impact: 'fatality' }  
  
Process "CNC Controller"  
└ executes → Function Asset "NC-Programm-Ausführung"  
  └ safety: { relevance: 'direct', impact: 'fatality',  
              rationale: 'Korrekte Ausführung des NC-Programms ist direkt  
sicherheitsrelevant' }
```

Ebene 2 – Asset-zu-Asset:

```
Function Asset "Not-Halt-Funktion"  
└ implemented_by → Process Asset "Not-Halt-Prozess"  
  
Function Asset "NC-Programm-Ausführung"  
└ depends_on → Data Asset "Fertigungsrezepte"  
  └ safety: { relevance: 'direct', impact: 'fatality' }
```

3.3 Process Assets

Identifizierte Process Assets: Zerspanungsprozess, Einrichtbetrieb, Not-Halt-Prozess, Qualitätsprüfung.

Beziehungen:

```
Process "CNC Controller"  
└ executes → Process Asset "Zerspanungsprozess"  
  
Process "SCADA System"  
└ monitors → Process Asset "Zerspanungsprozess"
```

```

EE "Operator"
└─ invokes → Process Asset "Einrichtbetrieb"

EE "Maintenance Technician"
└─ invokes → Process Asset "Einrichtbetrieb"

Process "Safety Controller (SIS)"
└─ executes → Process Asset "Not-Halt-Prozess"
  │   └─ safety: { relevance: 'direct', impact: 'fatality' }
└─ terminates → Process Asset "Zerspanungsprozess"
    └─ safety: { relevance: 'direct', impact: 'fatality',
                rationale: 'Not-Halt muss zuverlässig stoppen können' }

```

3.4 System Assets

Identifizierte System Assets: CNC-Maschine, Roboter, SCADA System, Safety Controller (SIS), MES.

Beziehungen:

```

Process "CNC Controller"
└─ is_an → System Asset "CNC-Maschine"
    └─ safety: { relevance: 'direct', impact: 'fatality',
                physicalHazardPotential: 'high' }

Process "Robot Controller"
└─ is_an → System Asset "Roboter"
    └─ safety: { relevance: 'direct', impact: 'fatality',
                physicalHazardPotential: 'high' }

Process "SCADA System"
└─ controls → System Asset "CNC-Maschine"
  │   └─ safety: { relevance: 'indirect', impact: 'fatality',
                  rationale: 'Kompromittiertes SCADA kann Maschinenbewegungen auslösen' }
└─ controls → System Asset "Roboter"
  │   └─ safety: { relevance: 'indirect', impact: 'fatality',
                  rationale: 'Kompromittiertes SCADA kann Roboterbewegungen auslösen' }
└─ monitors → System Asset "Safety Controller (SIS)"

Process "MES"
└─ controls → System Asset "SCADA System"

Process "VPN Gateway"
└─ uses [network] → System Asset "SCADA System"

Process "CNC Controller"
└─ depends_on → System Asset "SCADA System"

Process "Robot Controller"
└─ depends_on → System Asset "SCADA System"

Process "Safety Controller (SIS)"
└─ is_an → System Asset "Safety Controller (SIS)"
    └─ safety: { relevance: 'direct', impact: 'fatality' }

```

3.5 Infrastructure Assets

Identifizierte Infrastructure Assets: Schutzumhausung, Steuerungsschrank, CNC-Gehäuse, OT-Netzwerkinfrastruktur.

Beziehungen:

```

EE "Maintenance Technician"
├ accesses [on-site] → Infrastructure Asset "CNC-Maschine (physisch)"
├ accesses [internal] → Infrastructure Asset "Steuerungsschrank"
├ damages → Infrastructure Asset "Schutzumhausung"
  └ safety: { relevance: 'direct', impact: 'fatality',
              rationale: 'Physische Sabotage der Schutzumhausung hebt
Personenschutz auf' }

Process "Safety Controller (SIS)"
├ secures → Infrastructure Asset "Schutzumhausung"
  └ safety: { relevance: 'indirect', isPhysicalBarrier: true, impact: 'fatality',
              rationale: 'Verhindert Kontakt mit rotierenden Werkzeugen -
relevance: indirect da Schutzumhausung den Schaden nicht
steuert
sondern physisch unterbindet' }
├ monitors → Infrastructure Asset "Schutzumhausung"

Infrastructure Asset "CNC-Maschine (physisch)"
├ HighValue: {
  assetDestructionImpact: 'critical',
  isHighValueAsset: 'critical',
  isHighValueAssetSource: 'derived',
  replacementLeadTime: '>12m (critical)',
  highValueRationale: 'Speziell konfigurierte 5-Achsen-Anlage für
Titanbearbeitung.
Wiederbeschaffung, Rekonfiguration und
Wiederanlaufqualifizierung
erfordern > 12 Monate. Totalausfall = vollständiger
Produktionsstop.'
}

Infrastructure Asset "Steuerungsschrank"
├ HighValue: {
  assetDestructionImpact: 'high',
  isHighValueAsset: 'high',
  isHighValueAssetSource: 'derived',
  replacementLeadTime: '3-6m (medium)',
  highValueRationale: 'Kundenspezifische SPS-Konfiguration mit
applikationsspezifischer
Steuerungslogik. Standardhardware, aber Neukonfiguration,
Inbetriebnahme und Sicherheitsabnahme sind zeitintensiv.'
}

```

3.6 Human Assets

Identifizierte Human Assets: Maschinenbediener, Einrichter, NC-Programmierer, Wartungstechniker, Mitarbeiter (allgemein).

Unterscheidung Einrichter / NC-Programmierer: Der Einrichter testet und richtet Programme direkt an der Maschine ein – er ist physisch im Gefahrenbereich und damit Protection Target. Der NC-Programmierer erstellt Programme am Schreibtisch – er ist nicht physisch gefährdet, trägt aber Verantwortung für sicherheitskritische Logik.

Beziehungen:

```

EE "Operator"
├ is_an → Human Asset "Maschinenbediener"
  └ Properties: { isProtectionTarget: true, safetyImpact: 'fatality',
                  rationale: 'Physische Präsenz im Gefahrenbereich der Maschine' }

Process "CNC Controller"
├ affects_safety → Human Asset "Maschinenbediener"
  └ safety: { relevance: 'direct', impact: 'fatality',
              rationale: 'Unkontrollierte Werkzeugbewegung bei Fehlmanipulation' }

```

```

Process "Robot Controller"
└ affects_safety → Human Asset "Maschinenbediener"
  └ safety: { relevance: 'direct', impact: 'fatality',
             rationale: 'Kollision durch falsche Bewegungsbahn' }

Process "SCADA System"
└ exposes → Human Asset "Maschinenbediener"
  └ rationale: 'Kompromittierung ermöglicht Fernsteuerung der Maschine via SCADA'

EE "Maintenance Technician"
└ is_an → Human Asset "Einrichter"
  └ Properties: { isProtectionTarget: true, safetyImpact: 'fatality',
                 rationale: 'Physische Präsenz im Gefahrenbereich während
Einrichtung/Test' }

Process "CNC Controller"
└ affects_safety → Human Asset "Einrichter"
  └ safety: { relevance: 'direct', impact: 'fatality',
             rationale: 'Testablauf im reduzierten Schutzmodus – direkter
Gefahrenkontakt möglich' }

Process "Robot Controller"
└ affects_safety → Human Asset "Einrichter"
  └ safety: { relevance: 'direct', impact: 'fatality',
             rationale: 'Einrichter bewegt sich im Arbeitsbereich des Roboters –
unkontrollierte Bewegung bei reduziertem Schutzmodus
möglich' }

EE "NC Programmierer"
└ is_an → Human Asset "NC-Programmierer"
  └ Properties: { isProtectionTarget: false,
                 rationale: 'Erstellt NC-Programme am Schreibtisch – kein
physischer Gefahrenbereich' }

```

4. Asset Impact- und Schutzziel-Bewertung

Erst nachdem die Beziehungen im Graphen festgelegt sind, wird bewertet was jedes Asset schützenswert macht und warum. Die Safety-Relevanz ergibt sich dabei direkt aus den Beziehungen. Eine separate Analyse ist nicht erforderlich. Wer **affects_safety** auf einen Human Asset hat, bekommt Safety-Impact. Wer **secures** auf eine physische Barriere hat, ist sicherheitsrelevant.

4.0 Formale Aggregationslogik

Die aggregierte Asset-Kritikalität ergibt sich aus zwei unabhängigen Dimensionen: Business Impact und Physical Impact. Die Aggregation folgt einer expliziten Regel – keine Heuristik, keine Subjektivität.

Aggregationsmatrix:

Business Impact	Safety Impact	relevance	Aggregiert
beliebig	HIGH (fatality / irreversible_injury)	direct	CRITICAL
beliebig	HIGH (fatality / irreversible_injury)	indirect	HIGH+
CRITICAL	– oder MED	–	CRITICAL
HIGH	MED (indirect, Hop 1)	–	HIGH
HIGH	–	–	HIGH
MEDIUM	MED (indirect, Hop 1)	–	MEDIUM+
MEDIUM	–	–	MEDIUM
LOW	MED (indirect, Hop 1)	–	MEDIUM

Business Impact	Safety Impact	relevance	Aggregiert
LOW	–	–	LOW

Hinweis zu HIGH+/MEDIUM+: Diese Zwischenwerte bedeuten "oberes Ende der Stufe" und können durch hohe Likelihood in der Risk-Tabelle zu CRITICAL/HIGH eskalieren. Die finale Risikoeinstufung ergibt sich aus Impact × Likelihood – nicht aus dem aggregierten Kritikalitätswert allein.

Hinweis: **Physical Impact** wurde in **Safety Impact (physicalImpact)** umbenannt um die Konsistenz mit ISO 21434 Damage Categories und dem internen Derived/Manual Pattern herzustellen. HIGH entspricht **fatality** oder **irreversible_injury** aus der SafetyAnnotation, MED entspricht **indirect** (transitiv via Graph-Traversierung Hop 1).

Business Impact CRITICAL ist projektspezifisch zu definieren. Im NIS-2-Kontext z.B. bei schwerem volkswirtschaftlichem Schaden oder Verlust kritischer Infrastruktur, im CNC-Kontext z.B. bei totalem Produktionsausfall mit existenzbedrohenden Folgen. Safety-Aspekte führen zwar zwangsläufig zur Einstufung 'CRITICAL', sind jedoch bei weitem nicht der einzige Auslöser.

Safety Override Rule (formale Definition):

```

IF safetyImpact ∈ { 'fatality', 'irreversible_injury' }
  AND relevance === 'direct'
THEN aggregatedCriticality := CRITICAL
    strideDepth             := 'vertieft'
    riskPriority             := 1

IF safetyImpact ∈ { 'fatality', 'irreversible_injury' }
  AND relevance === 'indirect'
THEN aggregatedCriticality := HIGH+
    strideDepth             := 'fokussiert'
    riskPriority             := 2

```

Diese Regel implementiert das ISO 12100 Prinzip: physische Unversehrtheit von Menschen ist kein verrechenbarer Faktor. Die Differenzierung nach relevance stellt sicher dass nur Assets mit direkter Steuerungsverantwortung automatisch zu CRITICAL eskalieren. Assets mit indirektem Einfluss bleiben differenzierbar.

Diese Regel ist methodisch verbindlich und nicht überschreibbar.

4.0a Zwei Impact-Domänen und Safety-Ableitung

TARAflow bewertet Assets entlang zwei unabhängiger Impact-Domänen:

Domäne	Kriterien (Beispiele)
Business / Organisatorisch	Financial Damage, Regulatory/Compliance, Reputation, Operational Impact, Affected Users, Recoverability
Physical / Safety	Safety , Physical Asset Damage, Environmental Impact, Supply Chain

Die Kriterien sind projektspezifisch anpassbar und werden in Phase 0 (Projektprofil) festgelegt. Die Gesamtkritikalität pro Domäne ergibt sich nach dem Highest-Impact-Wins Prinzip.

Für diesen Referenzfall gewählte Impact-Faktoren:

Domäne	Faktor	Begründung
Business	Operational Impact	Produktionsausfall ist primäres Business-Risiko
Business	Financial Damage	Direkt aus Produktionsausfall ableitbar
Business	Regulatory / Compliance	IEC 62443, EU CRA, EN 50742
Business	Recoverability	Relevant für High-Value-Assets mit langen Wiederbeschaffungszeiten
Physical	Safety Impact	Fatality-Risiko ist dominierender Impact-Treiber

Domäne	Faktor	Begründung
Physical	Physical Asset Damage	High-Value-Assets: CNC-Maschine, Steuerungsschrank

Gewichtung der Impact-Faktoren (Phase 0 – Projektprofil):

Faktor	Gewicht	Begründung
Safety Impact	1.0	Maximum – entspricht Safety Override Rule; fatality + direct → immer CRITICAL unabhängig von Business Impact
Operational Impact	0.8	Primärer Business-Treiber – Produktionsausfall ist direkter und unmittelbarer Schaden
Recoverability	0.8	Multiplikator des Operational Impact – Wiederbeschaffungszeit > 12 Monate macht Ausfall existenzbedrohend
Financial Damage	0.6	Weitgehend aus Operational Impact ableitbar, aber explizit quantifizierbar (Vertragsstrafen, Umsatzausfall)
Regulatory / Compliance	0.6	Bussgelder und Marktzugangsverlust quantifizierbar; IEC 62443, EU CRA, EN 50742
Physical Asset Damage	0.3	Relevant primär für High-Value-Assets; für Data und Process Assets nicht anwendbar

Aggregationsmethode: Highest-Impact-Wins – der höchste gewichtete Einzelwert bestimmt den Business Impact. Niedrige Werte anderer Faktoren ziehen das Ergebnis nicht herunter. Diese konservative Methode stellt sicher dass kein kritischer Einzelfaktor durch den Durchschnitt anderer Faktoren maskiert wird.

Konfigurierbarkeit: Gewichte sind projektspezifisch in Phase 0 festgelegt. In regulierten Domänen (Medizintechnik, Bahn) wäre Regulatory/Compliance eher 0.8. In frühen Entwicklungsphasen ohne Normenpflicht eher 0.3.

Safety Annotation Pattern auf Beziehungsebene:

```
safety: {
  relevance: "none" | "indirect" | "direct";
  impact?: "none" | "reversible_injury" | "irreversible_injury" | "fatality";
  physicalHazardPotential?: "low" | "medium" | "high";
  protectionTarget?: boolean;           // nur bei Human Assets
  affectedSafetyFunctions?: string[];
  rationale?: string;
}
```

Ableitungsregel Safety Impact am Asset (Derived/Manual Pattern):

SafetyAnnotation auf Beziehung (relevance/impact)	→ physicalImpact am Asset
impact: 'fatality'	→ HIGH (derived)
impact: 'irreversible_injury'	→ HIGH (derived)
relevance: 'indirect' (transitiv, Hop 1)	→ MED (derived)
keine Annotation	→ LOW (derived, default)

Hop-Logik für transitive Safety-Relevanz:

```
Hop 0: Element —(Beziehung mit safety.relevance: 'direct')→ Asset
      → physicalImpact: HIGH (derived)
```

```
Hop 1: Element —(controls/depends_on)→ Element[Hop 0]
      → physicalImpact: MED (derived, relevance: 'indirect')

Hop 2+: → nicht automatisch propagiert (zu weit vom direkten Schadenspfad)
```

CNC-Beispiel:

```
Safety Controller (SIS) → is_an → SIS Asset → physicalImpact: HIGH (direct)
SCADA System → controls → Safety Controller → physicalImpact: MED
(indirect, Hop 1)
VPN Gateway → uses → SCADA System → nicht automatisch (Hop 2)
```

4.1 Begriffliche Präzisierung der CIANAAA-Schutzziele

Die Schutzziele Confidentiality, Integrity, Availability, Authentication und Authorization sind etabliert und werden im üblichen sicherheitstechnischen Sinn verwendet.

Die Ziele Non-Repudiation und Accountability werden in TARAflow bewusst unterschieden, da sie unterschiedliche Aspekte der Zurechenbarkeit beschreiben.

Non-Repudiation

Non-Repudiation bezeichnet die technische Nicht-Abstreitbarkeit einer Handlung. Eine Aktion muss so protokolliert und gesichert sein, dass sie einem Akteur eindeutig zugeordnet werden kann und nicht nachträglich bestritten werden kann.

Non-Repudiation entspricht dem R (Repudiation) in STRIDE und ist das Standard-Schutzziel überall wo Repudiation als Threat relevant ist.

Typische Massnahmen: Audit-Log, digitale Signatur, kryptografische Zeitstempel.

Accountability

Accountability bezeichnet die rechtliche und organisatorische Verantwortlichkeit für Handlungen und deren Konsequenzen. Sie geht über technisches Logging hinaus und adressiert die Nachweispflicht gegenüber Behörden und betroffenen Personen.

Accountability kommt zusätzlich zum Zuge wenn personenbezogene Daten verarbeitet werden (DSGVO Art. 5 Abs. 2) oder wenn eine organisatorische Verantwortlichkeit nachgewiesen werden muss.

Typische Kontexte: affects_privacy, tracks, HR-Systeme, Kundendaten, regulatorische Nachweispflichten gegenüber Behörden.

Ableitungsprinzip in TARAflow

Non-Repudiation wird abgeleitet überall wo der Beziehungstyp einen Repudiation-Threat ermöglicht. Das entspricht dem systematischen Vorgehen in STRIDE.

Accountability wird zusätzlich abgeleitet wenn das Asset einen Privacy-Bezug hat oder eine organisatorische Verantwortlichkeit gegenüber Behörden nachgewiesen werden muss.

Formal gilt:

```
Non-Repudiation: Standard bei Repudiation-relevanten Beziehungstypen
Accountability: Zusatz bei Privacy (DSGVO) und behördlicher Nachweispflicht
```

4.2 Schutzziel-Ableitung aus Beziehungstyp

Die CIANAAA-Schutzziele werden nicht frei vergeben. Sie folgen aus dem Beziehungstyp zwischen DFD-Element und Asset.

Data Assets:

Beziehungstyp	Primäre Schutzziele
stores	Integrity, Availability, Confidentiality*
reads	Confidentiality, Authorization
modifies	Integrity, Authorization, Non-Repudiation
creates	Integrity, Authentication, Non-Repudiation, Accountability**
destroys	Integrity, Authorization, Non-Repudiation, Accountability**
transports	Integrity, Confidentiality, Authentication, Accountability**

Process Assets:

Beziehungstyp	Primäre Schutzziele
is_an	Integrity, Availability, Confidentiality*
invokes	Authorization, Non-Repudiation, Accountability**
terminates	Authorization, Integrity
suspends	Authorization, Availability
monitors	Integrity, Non-Repudiation, Accountability**
executes	Integrity, Authorization, Non-Repudiation, Accountability**

Function Assets:

Beziehungstyp	Primäre Schutzziele
is_an	Integrity, Availability
executes	Integrity, Availability, Authorization
depends_on	Availability

System Assets:

Beziehungstyp	Primäre Schutzziele
is_an	Integrity, Availability
controls	Integrity, Availability, Authorization
configures	Integrity, Authorization, Non-Repudiation
depends_on	Availability
uses [network]	Authentication, Authorization, Integrity, Accountability**
uses [local]	Authorization, Integrity

Infrastructure Assets:

Beziehungstyp	Primäre Schutzziele
is_an	Integrity, Availability
accesses [on-site]	Authorization, Non-Repudiation
accesses [internal]	Authorization, Non-Repudiation
accesses [remote]	Authentication, Authorization, Non-Repudiation, Accountability**
secures	Availability, Integrity
powers	Availability

Beziehungstyp	Primäre Schutzziele
damages	Availability, Integrity
monitors	Integrity, Non-Repudiation, Accountability**

Human Assets:

Beziehungstyp	Primäre Schutzziele
is_an	– (Protection Target Definition)
affects_safety	Availability, Integrity
affects_privacy	Confidentiality, Authorization, Accountability
tracks	Confidentiality, Authorization, Accountability
exposes	Confidentiality, Authentication, Accountability**

** Confidentiality gilt nur wenn das Asset explizit als vertraulich klassifiziert ist.

*** Accountability gilt zusätzlich wenn das Asset oder der Kontext einen Personenbezug aufweist (DSGVO Art. 5 Abs. 2) oder eine behördliche Nachweispflicht besteht.

4.3 Data Assets

Asset	Schutzziele (CIANAAA)	Business Impact	Safety Impact	Aggregiert
Fertigungsrezepte	Integrity, Confidentiality, Authentication	HIGH (Operational)	fatality	CRITICAL
Kalibrierungsdaten	Integrity, Availability, Authorization	HIGH (Operational)	irreversible_injury	CRITICAL
Produktionsdaten	Availability, Confidentiality, Accountability	MEDIUM (Financial)	–	MEDIUM
Maschinenstatus	Integrity, Availability	MEDIUM (Operational)	indirect	HIGH
Diagnosedaten	Confidentiality, Authorization	LOW	–	LOW
Safety-Parameter	Integrity, Availability, Authorization	HIGH (Operational)	indirect	HIGH+
Auftragsdaten	Integrity, Availability	MEDIUM (Operational)	indirect	MEDIUM+

4.4 Function Assets

Asset	Schutzziele (CIANAAA)	Business Impact	Safety Impact	Aggregiert
Not-Halt-Funktion	Availability, Integrity	HIGH (Operational)	fatality	CRITICAL
NC-Programm-Ausführung	Integrity, Availability, Authorization	HIGH (Operational)	fatality	CRITICAL

4.5 Process Assets

Asset	Schutzziele (CIANAAA)	Business Impact	Safety Impact	Aggregiert
Zerspanungsprozess	Availability, Integrity, Authorization	HIGH (Operational)	fatality	CRITICAL
Not-Halt-Prozess	Availability, Integrity	HIGH (Operational)	fatality	CRITICAL

Asset	Schutzziele (CIANAAA)	Business Impact	Safety Impact	Aggregiert
Einrichtbetrieb	Integrity, Authorization, Accountability	MEDIUM (Operational)	indirect	HIGH+
Qualitätsprüfung	Integrity, Non-Repudiation	MEDIUM (Financial)	–	MEDIUM

4.6 System Assets

Asset	Schutzziele (CIANAAA)	Business Impact	Safety Impact	Aggregiert
CNC-Maschine	Availability, Integrity, Authorization	HIGH (Operational)	fatality	CRITICAL
Roboter	Availability, Integrity, Authorization	HIGH (Operational)	indirect	HIGH+
Safety Controller (SIS)	Availability, Integrity	HIGH (Operational)	fatality	CRITICAL
SCADA System	Availability, Authentication, Authorization	HIGH (Operational)	indirect	HIGH+
MES	Availability, Authentication, Authorization	HIGH (Operational)	indirect	HIGH+

4.7 Infrastructure Assets

Asset	Schutzziele (CIANAAA)	Business Impact	Safety Impact	Aggregiert	High-Value
Schutzumhausung	Availability, Integrity	HIGH (Operational)	direct	CRITICAL	–
Steuerungsschrank	Availability, Integrity, Authorization	HIGH (Operational)	indirect	HIGH+	✓ 3-6m
CNC-Maschine (physisch)	Availability, Integrity, Authorization	HIGH (Operational)	indirect	HIGH+	✓ >12m / CRITICAL

High-Value Assets: CNC-Maschine (physisch) und Steuerungsschrank sind als **isHighValueAsset** klassifiziert. CNC-Maschine: **isHighValueAsset: 'critical'** → **CRITICAL minimum** Override → Pflicht-Threats: Tampering, DoS, Physical Damage. Steuerungsschrank: **isHighValueAsset: 'high'** → **HIGH minimum** Override → Pflicht-Threats: Tampering, DoS, Physical Damage. Beide Werte folgen dem MINIMUM-Prinzip — Safety Override (fatality) hat weiterhin Vorrang.

Schutzumhausung neu CRITICAL: Die **damages**-Beziehung von **Maintenance Technician** ist als **direct** annotiert – physische Sabotage der Schutzumhausung hebt den Personenschutz direkt auf. Safety Override greift.

4.8 Human Assets

Asset	Schutzziele (CIANAAA)	Business Impact	Safety Impact	Protection Target	Aggregiert
Maschinenbediener	Availability (Safety)	–	fatality	✓	CRITICAL
Einrichter	Availability (Safety)	–	fatality	✓	CRITICAL
NC-Programmierer	Non-Repudiation, Accountability	MEDIUM (Operational)	–	–	MEDIUM

4.9 Ergebnis der Bewertung

Die Bewertung macht zwei Dinge sichtbar die ohne Graph nicht erkennbar wären:

Erstens: Von 24 identifizierten Assets sind 10 als CRITICAL eingestuft – wegen direktem Safety-Impact (relevance: 'direct'). Weitere 6 Assets sind als HIGH+ eingestuft da sie fatality oder irreversible_injury nur systemisch beeinflussen (relevance: 'indirect'). Die Differenzierung stellt sicher dass Priorisierung Trennschärfe behält: CRITICAL = direkter Steuerungseinfluss auf den Schaden, HIGH+ = systemischer Einfluss der durch hohe Likelihood zu CRITICAL eskalieren kann.

Zweitens: Die Function Assets (Not-Halt-Funktion, NC-Programm-Ausführung) sind neu als eigenständige CRITICAL Assets modelliert. Das macht sichtbar dass nicht nur die Prozesse und Systeme schützenswert sind, sondern die Capabilities selbst – insbesondere der Emergency Stop als Sicherheitsfunktion nach IEC 62443-3-3 SR 3.6.

5. Erkenntnissprung: Was TARAflow sichtbar macht

Der entscheidende Unterschied zu klassischen Methoden zeigt sich erst im direkten Vergleich. TARAflow macht Zusammenhänge sichtbar die bei elementbasierter Analyse verborgen bleiben.

5.1 Beispiel: NC-Programm Upload

Klassisch (STRIDE-naiv):

```
DF "NC-Programm Upload" → Tampering (möglich)
                        → Information Disclosure (möglich)
```

Kein Schadensbezug, keine Priorisierung, kein Safety-Kontext.

TARAflow:

```
DF "push nc program [cmd]"
└─ transports → Data Asset "Fertigungsrezepte"
    └─ Integrity: CRITICAL, safety: { impact: 'fatality' }
        └─ gelesen von: □ SCADA System → □ CNC Controller (via □ Network Share)
            └─ affects_safety → Human Asset "Maschinenbediener" [Protection Target]

→ Threat: Tampering auf NC-Programm
→ Konsequenz: Falsche Werkzeugbewegung → Personengefährdung
→ Priorität: CRITICAL (Safety Override Rule: fatality → immer höchste Priorität)
→ Massnahme: Digitale Signatur + Integritätsprüfung vor Programmausführung
```

5.2 Beispiel: Remote Support als versteckter Safety-Angriffsvektor

Klassisch:

```
EE "Remote Support" → uses → SCADA System
→ Spoofing (möglich), Tampering (möglich)
```

TARAflow:

```

EE "Remote Support"
└─ uses [network] via VPN Gateway → System Asset "SCADA System" [HIGH+]
    └─ controls → System Asset "CNC-Maschine" [CRITICAL, physicalHazardPotential:
high]
        └─ affects_safety → Human Asset "Maschinenbediener" [fatality]

→ Was vorher unsichtbar war:
Remote Support ist nicht nur ein IT-Risiko –
er ist ein transitiver Safety-Angriffsvektor.
Kompromittierter VPN-Zugang → Fernsteuerung via SCADA
→ Unkontrollierte Werkzeugbewegung → Personengefährdung

```

5.3 Beispiel: Safety Controller als Single Point of Safety-Failure

Klassisch: Safety Controller als normaler Prozess – STRIDE wie alle anderen.

TARFlow:

```

Process "Safety Controller (SIS)"
└─ executes → Function Asset "Not-Halt-Funktion" [Availability: CRITICAL, fatality]
└─ executes → Process Asset "Not-Halt-Prozess" [Availability: CRITICAL, fatality]
└─ terminates → Process Asset "Zerspanungsprozess"
└─ secures → Infrastructure Asset "Schutzumhausung" [isPhysicalBarrier]
└─ monitors → Infrastructure Asset "Schutzumhausung"

→ Was sichtbar wird:
Der Safety Controller ist der einzige Knoten der alle
Safety-kritischen Funktionen zusammenhält.
Denial of Service auf SIS = kein Not-Halt möglich = kein Schutz mehr.
Das ist ohne Asset-Beziehungen nicht erkennbar.

```

6. Asset-Priorisierung: Fokus durch Kritikalität

TARFlow analysiert nicht alle Elemente gleich tief. Die Asset-Kritikalität steuert den Analyseaufwand proportional zum potenziellen Schaden.

6.1 Kritikalitätsbewertung CNC-Assets

Asset	Business Impact	Physical Impact	Aggregiert	STRIDE-Tiefe
Fertigungsrezepte	HIGH (Operational)	fatality (Safety)	CRITICAL	Vertieft
Not-Halt-Funktion	HIGH (Operational)	fatality (Safety)	CRITICAL	Vertieft
Not-Halt-Prozess	HIGH (Operational)	fatality (Safety)	CRITICAL	Vertieft
CNC-Maschine (System)	HIGH (Operational)	fatality (Safety)	CRITICAL	Vertieft
Schutzumhausung	HIGH (Operational)	fatality (Safety)	CRITICAL	Vertieft
Kalibrierungsdaten	HIGH (Operational)	irreversible_injury	CRITICAL	Vertieft
SCADA System	HIGH (Operational)	indirect	HIGH+	Fokussiert
Safety-Parameter	HIGH (Operational)	indirect	HIGH+	Fokussiert
Produktionsdaten	MEDIUM (Financial)	–	MEDIUM	Hochstufig
Maschinenstatus	MEDIUM (Operational)	indirect	HIGH	Fokussiert
Diagnosedaten	LOW (Confidentiality)	–	LOW	Hochstufig

6.2 Entscheidungsmatrix STRIDE-Tiefe

Asset-Kritikalität	Trust Boundary	Attack Enabler	STRIDE-Tiefe	Beispiel CNC
CRITICAL	Ja	–	Vertieft	DF "push nc program [cmd]"
CRITICAL	Nein	–	Fokussiert	DataStore "NC-Programme" intern
HIGH	Ja	Ja	Fokussiert	SCADA System via VPN Gateway
HIGH	Nein	–	Fokussiert	Produktionsdaten intern
MEDIUM/LOW	Ja	–	Fokussiert	Diagnosedaten via VPN
MEDIUM/LOW	Nein	–	Hochstufig	Maschinenstatus intern

6.3 Safety Override Rule

Assets mit `safetyImpact: 'fatality'` oder `safetyImpact: 'irreversible_injury'` erhalten automatisch die höchste Analysepriorität, unabhängig vom Business Impact. Ein Menschenleben ist nicht mit wirtschaftlichem Schaden verrechenbar (ISO 12100).

```

Beispiel Safety Override:
Diagnosedaten → Business Impact: LOW → STRIDE-Tiefe: Hochstufig (normal)

Not-Halt-Funktion → Business Impact: HIGH
                    → Physical Impact: fatality
                    → STRIDE-Tiefe: Vertieft (Safety Override greift!)

```

7. STRIDE-Analyse (Auszug)

Die folgenden Threats zeigen wie Asset-Beziehungen die STRIDE-Analyse direkt steuern. Jeder Threat ist vollständig rückverfolgbar bis zum Asset und seiner Safety-Konsequenz.

Threat T-01: Manipulation NC-Programm

Feld	Inhalt
DFD-Element	DF <code>push nc program [cmd]</code>
Asset-Beziehung	<code>transports</code> → Data Asset "Fertigungsrezepte"
STRIDE-Kategorie	Tampering
CIANAAA-Verletzung	Integrity, Authentication
Schutzziel	Fertigungsrezepte dürfen nicht unautorisiert verändert werden
Angriffspfad	NC Programmer Workstation kompromittiert → manipuliertes NC-Programm ins MES hochgeladen → SCADA System liest falsches Programm → CNC Controller führt falsche Bewegungssequenz aus
Safety-Konsequenz	<code>affects_safety</code> → Human Asset "Maschinenbediener" → fatality
Priorität	CRITICAL (Safety Override Rule)
STRIDE-Tiefe	Vertieft (Trust Boundary CNA → ERP + CRITICAL Asset)

Threat T-02: Kompromittierung Remote-Zugang

Feld	Inhalt
DFD-Element	EE Remote Support via DF <code>pull scada diagnostics ext [req]</code>

Feld	Inhalt
Asset-Beziehung	uses [network] via VPN Gateway → System Asset "SCADA System"
STRIDE-Kategorie	Spoofing + Elevation of Privilege
CIANAAA-Verletzung	Authentication, Authorization
Schutzziel	Nur autorisierter Remote-Zugriff auf Steuerungsfunktionen
Angriffspfad	VPN-Credentials kompromittiert → Angreifer übernimmt Remote-Support-Rolle → Zugang zu SCADA System → Fernsteuerung CNC-Achsen möglich
Safety-Konsequenz	exposes → Human Asset "Maschinenbediener" → fatality
Priorität	CRITICAL (Safety Override Rule)
STRIDE-Tiefe	Vertieft (Trust Boundary ext → CNA + CRITICAL Asset + Safety)

Threat T-03: Denial of Service Safety Controller

Feld	Inhalt
DFD-Element	Process Safety Controller (SIS)
Asset-Beziehung	executes → Function Asset "Not-Halt-Funktion"
STRIDE-Kategorie	Denial of Service
CIANAAA-Verletzung	Availability
Schutzziel	Not-Halt-Funktion muss jederzeit verfügbar und auslösbar sein
Angriffspfad	Netzwerk-Flooding auf OT-Segment → Safety Controller nicht mehr erreichbar → Not-Halt kann nicht ausgelöst werden → Maschine läuft ohne Safety-Funktion
Safety-Konsequenz	terminates → Process Asset "Zerspanungsprozess" nicht mehr möglich → fatality
Priorität	CRITICAL (Safety Override Rule)
STRIDE-Tiefe	Vertieft (CRITICAL Asset + Safety)

Threat T-04: Unbefugter physischer Zugang Steuerungsschrank

Feld	Inhalt
DFD-Element	EE Maintenance Technician
Asset-Beziehung	accesses [internal] → Infrastructure Asset "Steuerungsschrank"
STRIDE-Kategorie	Tampering
CIANAAA-Verletzung	Integrity, Availability, Authorization, Accountability
Schutzziel	Physischer Zugang zu Steuerungshardware nur für autorisiertes Personal
Angriffspfad	Ungesicherter Schrank → direkter Hardwarezugang → Manipulation SPS-Konfiguration oder Hardware-Implant einschleusen
Safety-Konsequenz	indirect via System Asset "CNC-Maschine" → fatality
Priorität	HIGH

Feld	Inhalt
STRIDE-Tiefe	Fokussiert
Threat T-05: Sabotage Schutzumhausung	
Feld	Inhalt
DFD-Element	EE Maintenance Technician
Asset-Beziehung	damages → Infrastructure Asset "Schutzumhausung"
STRIDE-Kategorie	Tampering
CIANAAA-Verletzung	Integrity, Availability
Schutzziel	Physische Integrität der Schutzumhausung als Personenschutzbarriere
Angriffspfad	Insider-Angriff → physische Manipulation der Schutzumhausung → Schutzfunktion ausgehebelt → Maschinenbereich zugänglich während Betrieb
Safety-Konsequenz	endangers → Human Asset "Maschinenbediener" → fatality
Priorität	CRITICAL (Safety Override Rule: direct + fatality)
STRIDE-Tiefe	Vertieft

8. Risk-Tabelle (Auszug)

ID	Threat	Likelihood	Impact	Risk	Safety Override	Massnahme	Prio
T-01	NC-Programm Manipulation	MEDIUM	CRITICAL	HIGH	✓ fatality	Digitale Signatur NC-Programme, Integritätsprüfung vor Ausführung	1
T-02	Remote-Zugang kompromittiert	HIGH	CRITICAL	CRITICAL	✓ fatality	MFA, VPN mit Zertifikaten, Session-Monitoring, Least Privilege	1
T-03	DoS Safety Controller	MEDIUM	CRITICAL	HIGH	✓ fatality	Netzwerksegmentierung SIS, dediziertes Safety-Netzwerk, Redundanz	1
T-04	Physischer Zugang Schrank	LOW	HIGH	MEDIUM	–	Schloss mit Zugangskontrolle, Versiegelung, Zugangsprotokoll	2
T-05	Sabotage Schutzumhausung	LOW	CRITICAL	HIGH	✓ fatality	Manipulationsschutz, Sensor-Überwachung, Zugangsprotokoll	1
T-06	Manipulation Kalibrierungsdaten	LOW	CRITICAL	HIGH	✓ irreversible_injury	Schreibschutz, Versionierung, Änderungsprotokoll mit Signatur	1

Safety Override Rule in der Praxis: T-01 hat Likelihood MEDIUM – klassisch wäre das Risk HIGH mit normaler Priorität. Durch die Safety Override Rule (fatality) wird es zu Priorität 1 unabhängig vom Business Impact. Diese Entscheidung ist methodisch begründet und auditierbar nach ISO 12100.

9. Der TARAflow-Graph als Single Source of Truth

Alle Asset-Beziehungen bilden zusammen einen gerichteten Graphen:

Knoten: DFD-Elemente (Process, DataStore, EE, DF) + Assets (6 Kategorien)
Kanten: typisierte Beziehungen (controls, depends_on, affects_safety, ...)

Dieser Graph ist die **einzige Datenquelle** – alle Diagramme und Dokumente werden daraus abgeleitet. Es gibt keine Redundanz, keine Inkonsistenz zwischen verschiedenen Dokumenten. Eine Änderung im Modell propagiert in alle Sichten gleichzeitig.

9.1 Der Graph als Analyse-Engine

Der entscheidende Unterschied zu einem Modellierungswerkzeug ist: TARAflow leitet Ergebnisse **regelbasiert** aus dem Graphen ab. Die Kanten sind formal typisiert – das ermöglicht regelbasierte Ableitungen ohne manuelle Interpretation.

Regelbasierte Ableitungen (methodisch definiert, keine Analyst-Entscheidung):

Auslöser im Graph	Automatische Ableitung
DF mit transports + Asset Integrity: HIGH+	Tampering als Pflicht-Threat
DF mit transports + Asset Confidentiality: HIGH+	Information Disclosure als Pflicht-Threat
EE mit uses[network] + Trust Boundary	Spoofing als Pflicht-Threat
Process mit is_an + Asset Availability: CRITICAL	DoS als Pflicht-Threat
DF mit is_an + Function Asset + Safety direct	Tampering + DoS als Pflicht-Threat
Asset mit safetyImpact: fatality	riskPriority := 1, strideDepth := vertieft
Asset mit safetyImpact: fatality + Human Asset isProtectionTarget	Safety-Sicht Eintrag, Safety-Kette Dokumentation
DF überschreitet Trust Boundary ohne parentConduit	Validierungsfehler (ERROR)
SafetyAnnotation impact: 'fatality' auf Beziehung	physicalImpact := HIGH (derived)
SafetyAnnotation impact: 'irreversible_injury' auf Beziehung	physicalImpact := HIGH (derived)
Element Hop-1 zu Asset mit physicalImpact HIGH	physicalImpact := MED (derived, indirect)
physicalImpact manuell gesetzt ohne DFD-Annotation	Validierungswarning + Rationale-Pflicht
physicalImpactSource === 'manual' im Audit-Export	Rationale verbatim dokumentiert

Heuristische Ableitungen (Analyst-Entscheidung, durch Modell informiert aber nicht erzwungen):

Entscheidung	Basis im Modell	Analyst-Anteil
Likelihood-Bewertung	Expositionsbewertung, Angreiferprofile (kontextabhängig)	Hoch
Attack Enabler Identifikation	Graph-Traversierung auf indirekte Pfade	Mittel
Massnahmen-Auswahl	Threat-Kategorie + Asset-Kritikalität	Mittel
Systemgrenze und Scope	Projektkontext	Hoch

Konsequenz: TARAflow ist keine vollständig automatische Analyse – das wäre methodisch nicht seriös. Aber der regelbasierte Methodikkern stellt sicher dass keine Pflicht-Threats übersehen werden und alle Safety-kritischen Pfade vollständig dokumentiert sind.

10. Automatisch generierbare Diagramme

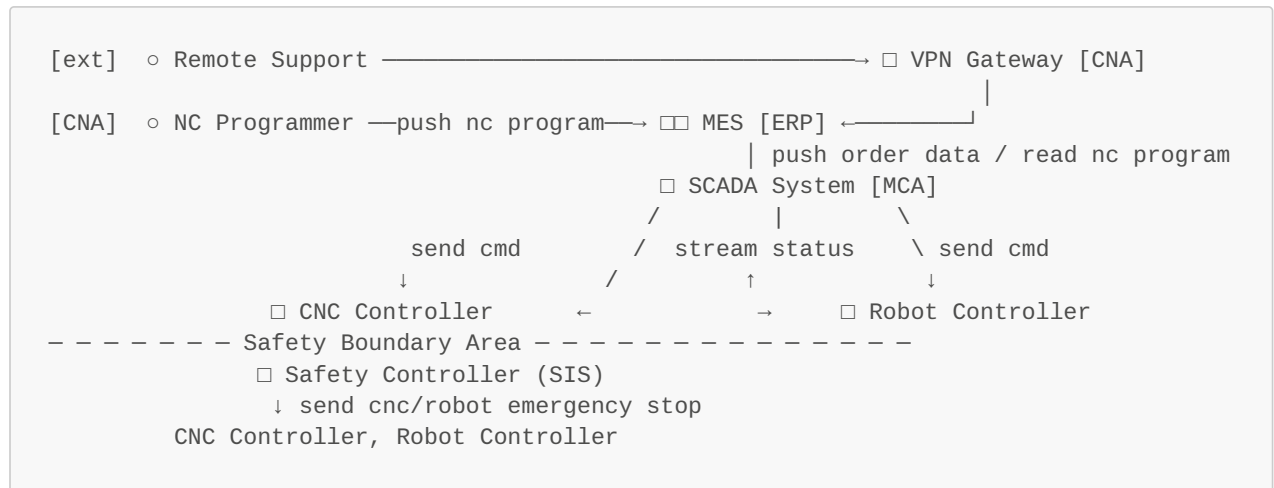
Die folgenden Sichten sind Filterabfragen auf denselben Graphen. Im DFD-Tab von TARAflow sind sie über ein Dropdown direkt zugänglich.

10.1 DFD (Standard-Sicht)

Filter: Alle DFD-Elemente und Datenflüsse

Zielgruppe: Security Engineer, Entwickler

Zeigt: Systemstruktur, Datenflüsse, Trust Boundaries

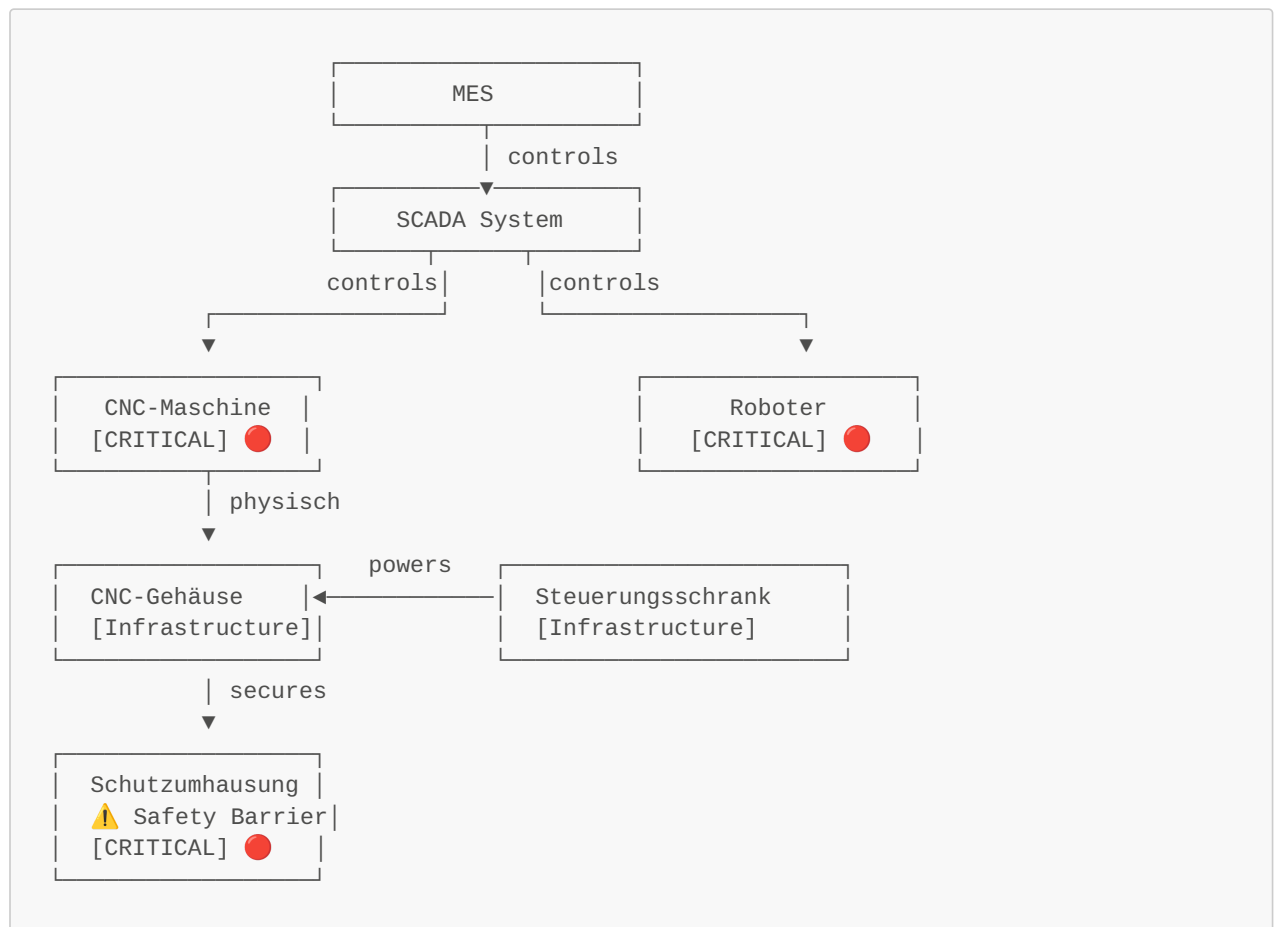


10.2 Systemarchitektur-Sicht

Filter: System Assets + Infrastructure Assets; Kanten: controls, depends_on, powers, secures

Zielgruppe: Systemarchitekt, Management, Auditor

Zeigt: Komponenten und ihre Abhängigkeiten, physische Infrastruktur



10.3 Safety-Sicht

Filter: Nur safety-annotierte Knoten und Kanten; Hervorhebung von Protection Targets

Zielgruppe: Safety Engineer, Auditor (EN 50742, ISO 12100)

Zeigt: Alle sicherheitsrelevanten Pfade zum Menschen als Schutzobjekt

SAFETY-KRITISCHE PFADE

● = fatality risk

- CNC Controller —affects_safety→ 👤 Maschinenbediener ●
- Robot Controller —affects_safety→ 👤 Maschinenbediener ●
- SCADA System —exposes→ 👤 Maschinenbediener ●
- 🔧 Schutzumhausung —[Safety Barrier]→ 👤 Maschinenbediener ●

Kritische Kette (T-01):

○ NC Programmer → □ MES → 📁 Network Share → □ SCADA System → □ CNC Controller
└

affects_safety → 👤 ●

Kritische Kette (T-02):

○ Remote Support → □ VPN Gateway → □ SCADA System → □ CNC Controller
└ affects_safety → 👤 ●

Kritische Kette (T-05):

○ Maintenance Technician —damages→ 🔧 Schutzumhausung
└ [Safety Barrier versagt] → 👤 ●

10.4 Angriffsflächen-Sicht

Filter: External Entities + ihre Verbindungen; Kanten: uses[, accesses]

Zielgruppe: Security Engineer, Penetration Tester

Zeigt: Einstiegspunkte und Angriffsvektoren mit Bewertung

- Remote Support —uses[network]→ □ VPN Gateway → □ SCADA System ⚠️
CRITICAL
- NC Programmer —push→ □ MES → 📄 Fertigungsrezepte ⚠️
CRITICAL
(Safety-kritisch!)
- Operator —————→ □ SCADA System ⚠️ HIGH
- Maintenance Technician —accesses[on-site]→ 🔧 CNC-Maschine (physisch) ⚠️ HIGH
—accesses[internal]→ 🔧 Steuerungsschrank ⚠️ HIGH
—damages→ 🔧 Schutzumhausung ⚠️
CRITICAL

Zusammenfassung: 4 EEs | 8 Vektoren | 4 CRITICAL mit Safety-Relevanz

10.5 Datenpfad-Sicht

Filter: Data Assets + ihre Träger; Kanten: creates, reads, modifies, transports

Zielgruppe: Security Engineer, Datenschutzbeauftragter

Zeigt: Datenflüsse, Kritikalität und Zugriffsmuster

- 📄 Fertigungsrezepte [Integrity: CRITICAL] ●
 - creates: ○ NC Programmer (via MES)
 - stores: 📁 Network Share, 📁 CNC Filesystem
 - transports: DF "push nc program [cmd]"
 - reads: □ SCADA System, □ CNC Controller
- 📄 Kalibrierungsdaten [Integrity: CRITICAL] ●
 - stores: 📁 CNC Filesystem
 - reads: □ CNC Controller
 - modifies: ○ Maintenance Technician (via SCADA System)

```

📄 Maschinenstatus [Availability: HIGH]
  creates:   □ CNC Controller
  transports: DF "stream machine status [stream]"
  reads:     □ SCADA System

📄 Produktionsdaten [Availability: MEDIUM]
  creates:   □ SCADA System
  stores:    🗄 Historian Database

```

10.6 Abhängigkeits-Sicht

Filter: System + Process Assets; Kanten: depends_on

Zielgruppe: Management, Business Continuity

Zeigt: Kaskadeneffekte bei Ausfall, Single Points of Failure

```

□ CNC Controller + Robot Controller
└─ depends_on → □ SCADA System
    └─ Ausfall → gesamte Fertigung steht (Business Critical)

□ CNC Controller + Robot Controller
└─ depends_on → □ Safety Controller (SIS)
    └─ Ausfall → kein Not-Halt möglich (SAFETY CRITICAL) ●

□ SCADA System
└─ depends_on → □□ MES
    └─ Ausfall → keine Auftragssteuerung, kein NC-Programm Upload

Single Points of Failure:
  □ SCADA System          → Business Critical (Produktionsstillstand)
  □ Safety Controller (SIS) → Safety Critical (kein Personenschutz) ●

```

11. Warum ein Graph – nicht viele Dokumente

Der zentrale Vorteil von TARAflow liegt in der **einmaligen Modellierung** mit **mehrfacher Nutzung**:

Deliverable	Manuelle Erstellung	TARAflow
DFD	Manuell zeichnen	✓ Eingabe-Diagramm
Systemarchitektur	Separat erstellen	✓ Automatisch aus Graph
Safety-Nachweis (EN 50742)	Manuell dokumentieren	✓ Automatisch aus Graph
Asset-Inventar (IEC 62443)	Separat pflegen	✓ Automatisch aus Graph
Angriffsflächen-Analyse	Separate Analyse	✓ Automatisch aus Graph
TARA-Dokumentation	Manuell erstellen	✓ Kern-Output
Risk-Tabelle	Manuell erstellen	✓ Aus Threat-Analyse
Abhängigkeitsanalyse	Separate FMEA	✓ Automatisch aus Graph

Konsequenz: Eine Änderung im Modell propagiert automatisch in alle Sichten und Dokumente. Kein Synchronisationsaufwand, keine Inkonsistenz, kein Mehraufwand bei Systemänderungen.

12. Safety-Integration als Alleinstellungsmerkmal

Das CNC-Beispiel zeigt deutlich: Safety und Security sind nicht trennbar. TARAflow ist das einzige Threat-Modeling-Framework das diese Kette durchgängig und explizit modelliert.

Die vollständige Safety-Kette im Graphen

```
Angriff auf NC-Programm (Cyber-Bedrohung)
  ↓ transports [Integrity verletzt]
Manipulation Fertigungsrezepte (Data Asset)
  ↓ reads
Fehlerhafter Bewegungsablauf (Function Asset: NC-Programm-Ausführung)
  ↓ affects_safety
Unkontrollierte Maschinenbewegung (System Asset: CNC-Maschine)
  ↓ secures [Schutzfunktion versagt]
Versagen Schutzumhausung (Infrastructure Asset)
  ↓ exposes
Körperverletzung / Tod (Human Asset - Protection Target) ●
```

Jeder Schritt dieser Kette ist im Graph explizit modelliert. Daraus folgt automatisch die höchste Threat-Priorität – begründet, auditierbar, normkonform.

Safety Override Rule

Assets mit **safetyImpact: 'fatality'** erhalten automatisch die höchste Risikopriorität – unabhängig von Business Impact oder Likelihood. Ein Menschenleben ist nicht mit wirtschaftlichem Schaden verrechenbar (ISO 12100 Prinzip). TARAflow setzt diese Regel automatisch durch und dokumentiert sie für den Audit.

13. Positionierung gegenüber anderen Tools

Kriterium	Klassisch STRIDE	IriusRisk	TARAflow
Ansatz	Element → generische Threats	Regelbasiert (Threat Library)	Graphbasiert (Asset → Beziehungen → Threats)
Safety-Integration	Nicht vorhanden	Nicht vorhanden	Kern-Feature
Rückverfolgbarkeit	Keine	Eingeschränkt	Vollständig
Priorisierung	Manuell, subjektiv	Regelbasiert	Asset-Kritikalität + Safety Override
Diagramm-Generierung	Manuell	Manuell	Automatisch aus Graph
Norm-Fokus	Keine	IEC 62443-2-x	IEC 62443-4-1/4-2, CRA
Zielgruppe	Alle	Integratoren	Hersteller, Consultants

14. Nächste Schritte mit diesem Referenzfall

1. **Graph vervollständigen** – alle Asset-Beziehungen formal definieren und in TARAflow erfassen
2. **STRIDE vollständig** – alle Threats für CRITICAL und HIGH Assets systematisch ableiten
3. **Attack Trees** – für T-01 und T-02 als Beispiel für Phase 3 TARAflow
4. **Massnahmen vollständig** – alle Threats mit konkreten Massnahmen und Normnachweis
5. **Export** – alle Diagramme und Dokumente automatisch aus dem Graph generieren
6. **Vortrag** – Referenzfall als Live-Demo in TARAflow mit allen Diagramm-Sichten

Dieser Referenzfall ist der primäre Testfall für alle TARAflow-Features und dient als Demonstrationsgrundlage für Kunden, Consultants und Auditoren.